

RISC-V Zvdota/Zvbdota扩展介绍

陈嘉炜 6.17
jiawei@iscas.ac.cn

报告内容

- Zvdota/Zvbdota扩展背景与设计
 - Horizontal dot-product 语义
 - batched dot-product
 - 数据格式、vtype.altfmt、ci immediate
- GNU工具链支持方案
 - Binutils 支持: -march、opcode、operand、tests
 - GCC 支持: machine pattern、RVV intrinsic、合法性检查
- Upstream进展与后续工作

什么是Zvdota/Zvbdota

RISC-V 向量点积(dot-product)扩展集合的简称。

它分为两类向量点积运算——

Zvdota —— 向量点积运算 $c = \sum_i \vec{a}_i \cdot \vec{b}_i$ (结果是一个标量)

Zvbdota —— 批量点积运算 $\vec{c}_j = \sum_i \vec{a}_i \cdot \vec{b}_{j,i}$ (结果是一个向量)

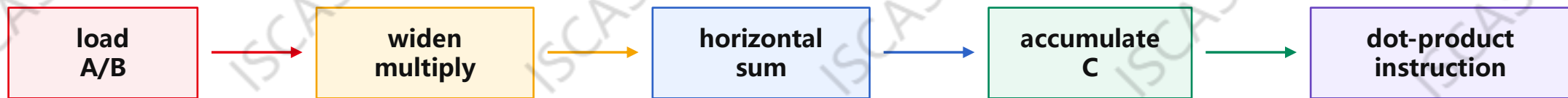
草案地址:

<https://github.com/riscv/riscv-isa-manual/tree/zvbdot>

为什么需要 Dot-Product 指令？

核心场景——AI 推理/训练、DSP、GEMM/Conv 都以向量内积为核心指令操作。

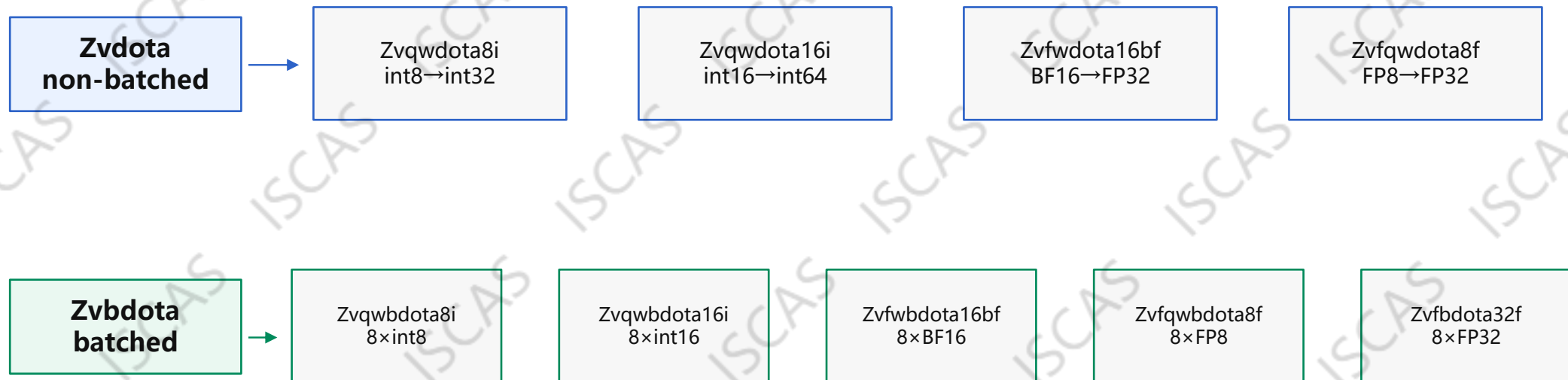
核心收益——减少 multiply / widening / reduction / accumulate 指令序列，提升寄存器复用。



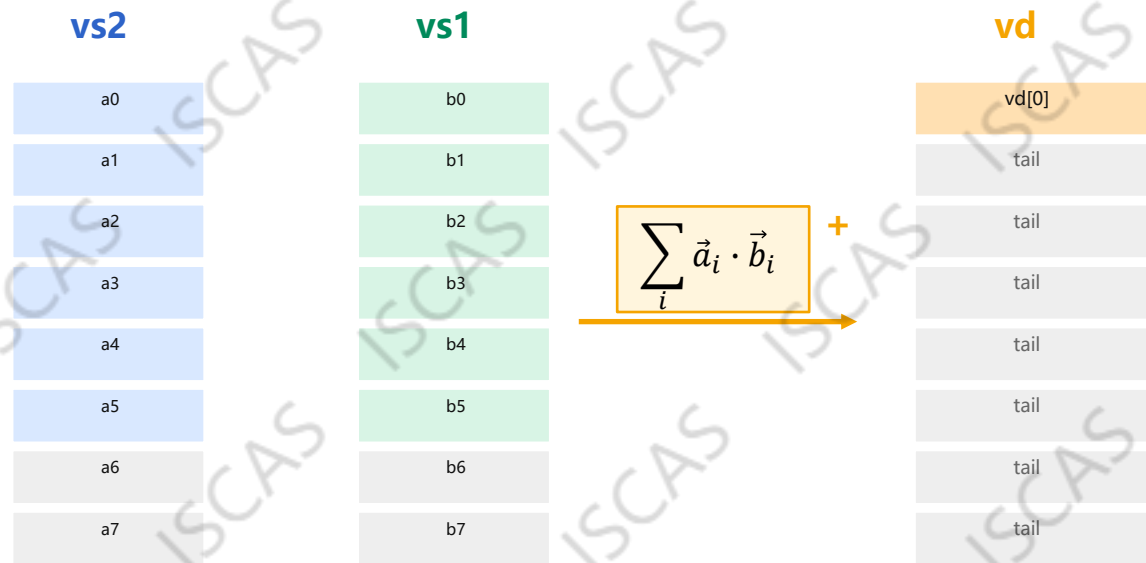
数据类型	典型用途	累加宽度
int8	量化推理 / int8 GEMM	int32
int16	DSP / fixed-point / 通信	int64
BF16	AI 训练/推理	FP32
FP8	Transformer / mixed precision	FP32
FP32	SGEMM / HPC fallback	FP32

Zvdota/Zvbdota扩展系列

核心区别：Zvdota 产生 1 个 dot-product result；Zvbdota 一次产生 8 个 dot-product results，更贴近 GEMM micro-kernel.



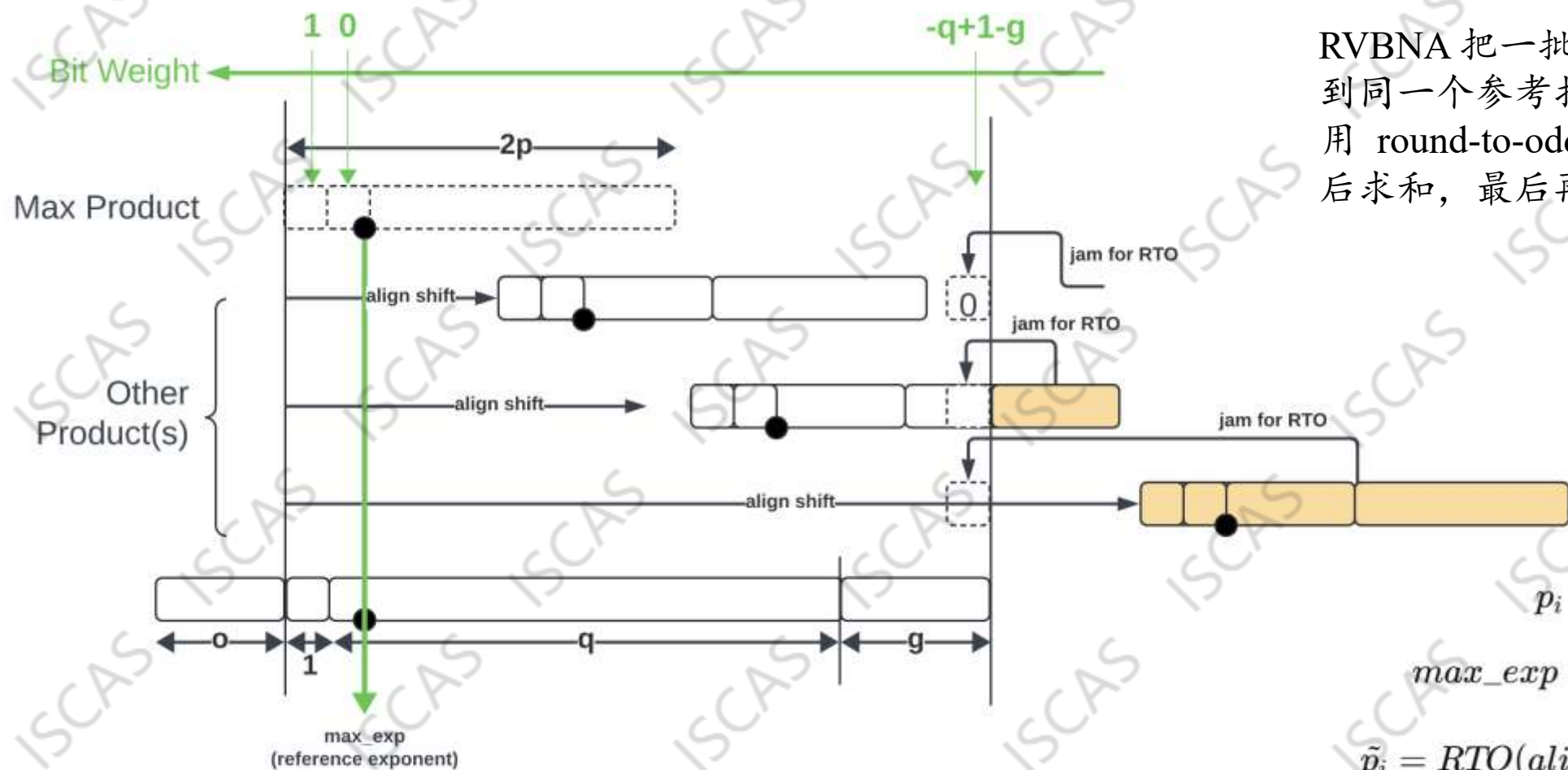
Zvdota计算Horizontal dot&accumulation



```
vqwdotau.vv vd, vs2, vs1, vm  
vqwdotas.vv vd, vs2, vs1, vm  
vfwdotau.vv vd, vs2, vs1, vm  
vfqwdota.vv vd, vs2, vs1, vm
```

- 指令约束
 - 目标寄存器组 vd 不得与源寄存器组地址重叠
 - vstart $\neq 0$ 为保留状态
 - vd[0] 保存累加结果，vd 其他元素按 tail policy 处理
- 运算规则
 - 整数版本：采用模累加（modulo accumulation）语义
 - BF16/FP8 浮点版本：使用 RVBNA 批量归一化（bulk normalization）算法
- 场景与适配说明
 - 适用场景：单输出的点积规约（dot-product reduction）运算
 - 适配局限：与 GCC 现有逐通道计算（lane-wise）DOT_PROD_EXPR 语义不匹配，目前无法直接复用自动向量化框架

RISC-V批量归一化算法(RVBNA)



RVBNA 把一批浮点乘积先对齐到同一个参考指数 max_exp , 再用 round-to-odd 做中间舍入, 然后求和, 最后再舍入到目标精度。

$$p_i = a_i b_i$$

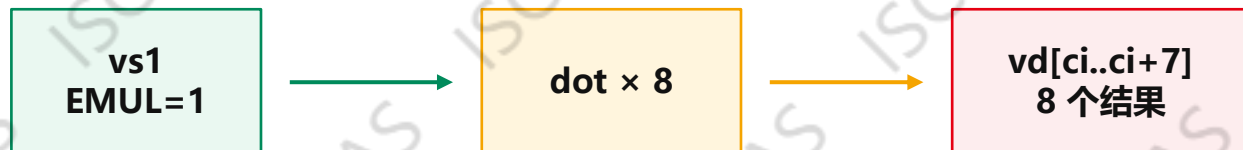
$$\text{max_exp} = \max_i \text{exp}(p_i)$$

$$\tilde{p}_i = \text{RTO}(\text{align}(p_i, \text{max_exp}))$$

$$\text{result} = \text{RTO}\left(\sum_i \tilde{p}_i + \text{acc}\right)$$

Zvbdota: 一次计算 8 个 dot products

面向 GEMM micro-kernel: 1个 vs1 向量与 vs2 中 8 个子向量分别做 dot-product, 写回 vd 中 8 个 accumulator。

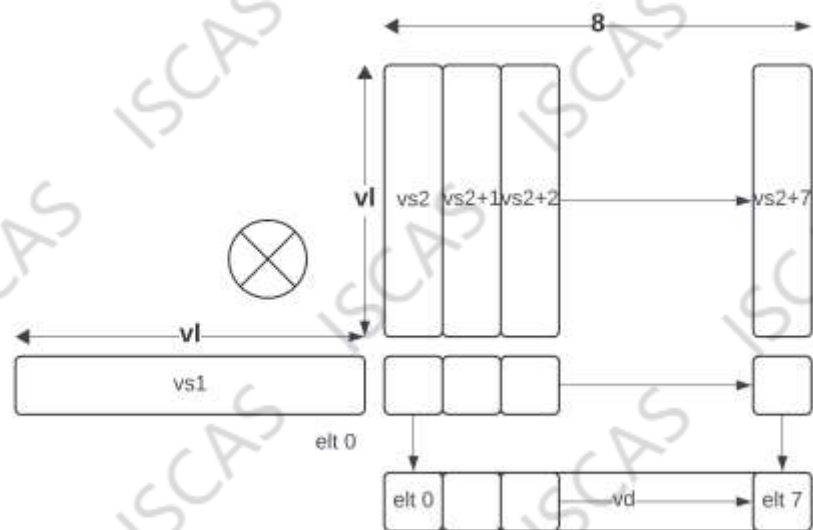


vs2 group: EMUL=8



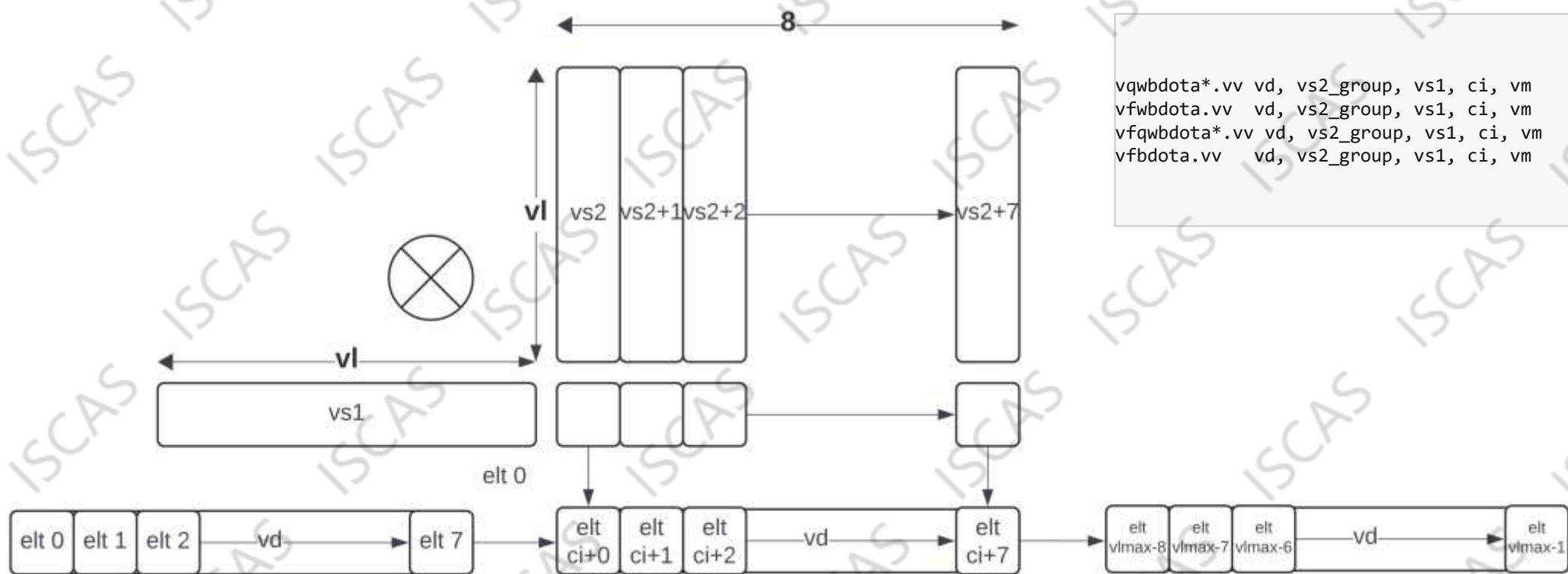
v0-v7 / v8-v15 / ...

```
vqwbdota*.vv vd, vs2_group, vs1, ci, vm
vfwbdota.vv vd, vs2_group, vs1, ci, vm
vfqwbdota*.vv vd, vs2_group, vs1, ci, vm
vfbdota.vv vd, vs2_group, vs1, ci, vm
```



Zvbdota: $VLEN > 8 * EEW$ 时使用 ci 进行偏移

当 VLEN 比较大时，vd 里可能有 16、32、甚至更多个 FP32/Int32 accumulator 元素。但 Zvbdota 每条指令一次只更新 8 个 dot-product 结果。那就需要一个偏移量来指定要更新 vd 中哪 8 个连续元素。



vtype.altfmt: 把“第二个维度”放进 vsetvli

在 Zvdota/Zvbdota 中，mnemonic 控制 vs2，vtype.altfmt 控制 vs1.

场景	mnemonic / .alt 控制	vtype.altfmt 控制	vsetvli 表达
整数	vs2 unsigned/signed	vs1 unsigned/signed	e8 / e8alt, e16 / e16alt
BF16	固定 BF16 dot	要求 altfmt=1	e16alt
FP8	vs2 E4M3 / E5M2	vs1 E4M3 / E5M2	e8 / e8alt
FP32 vfbdots	普通 FP32, 不使用 altfmt	使用 FRM / rounding-mode 处理 vtype 为 e32,m1	e32,m1 + FRM

整数映射示例

vs2	vs1	asm mnemonic	altfmt
unsigned	unsigned	vqwdotau	0
unsigned	signed	vqwdotau	1
signed	unsigned	vqwdotas	0
signed	signed	vqwdotas	1

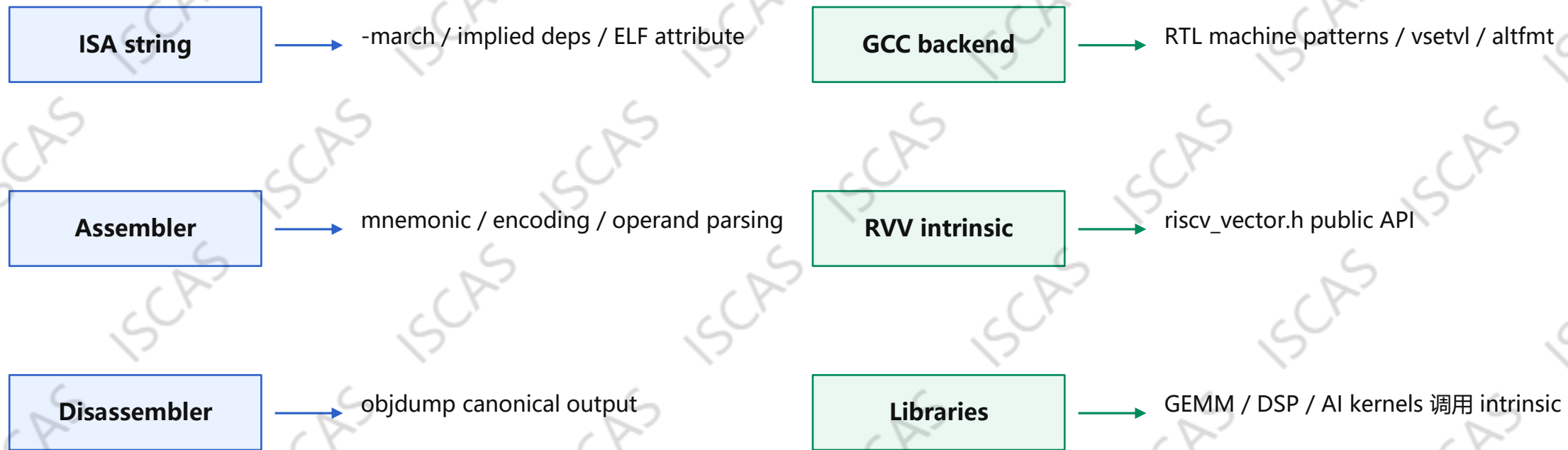
FP8 映射示例

vs2	vs1	asm mnemonic	altfmt
E4M3	E4M3	vfqwdota	0
E4M3	E5M2	vfqwdota	1
E5M2	E4M3	vfqwdota.alt	0
E5M2	E5M2	vfqwdota.alt	1

- Zvdota/Zvbdota扩展背景与设计
 - Horizontal dot-product 语义
 - batched dot-product
 - 数据格式、vtype.altfmt、ci immediate
- GNU工具链支持方案
 - Binutils 支持: -march、opcode、operand、tests
 - GCC 支持: machine pattern、RVV intrinsic、合法性检查
- Upstream进展与后续工作

工具链支持目标：先显式可用，再考虑自动化

实现优先级：让手写汇编、intrinsics 和库 kernel 能稳定使用；自动向量化后续单独设计。



当前系列不做 autovec

Binutils 实现（一）：arch string 与隐式依赖

目标：gas 接受扩展名，ELF attribute 能记录能力，链接/反汇编行为一致。

改动位置

- bfd/elfxx-riscv.c: riscv_supported_std_z_ext / riscv_implicit_subsets
- include/opcode/riscv.h: 新增 `INSN_CLASS_*`
- gas/testsuite/gas/riscv/march-help.l: 扩展列表测试
- gas/NEWS: 用户可见支持说明

依赖展开

```
zvqwdota8i    -> +zve32x
zvqwdota16i   -> +zve64x
zvfwdota16bf  -> +zve32f
zvfqwdota8f   -> +zve32f

zvdota        -> all Zvdota family extensions
zvbdota       -> all Zvbdota family extensions
```

测试目标	示例
主覆盖测试	-march=rv64gc_zvdota
单独 gating	-march=rv64gc_zvqwdota16i
属性检查	readelf -A / objdump -dr

Binutils 实现（二）：opcode table 与特殊 operand

Zvdota —— 普通 vector register operands: Vd,Vt,VsVm, 对应 vd,vs2,vs1,mask

Zvbdota —— 新增 vs2_group + ci: Vq 负责 vs2[4:3], Vr 负责 vs2[2:0]

改动位置 opcodes/riscv-opc.c riscv-dis.c gas/config/tc-riscv.c

```
{"vqwbdotas.vv", 0, INSN_CLASS_ZVQWBDOTA8I_OR_ZVQWBDOTA16I,
 "Vd,Vq,Vs,VrVm", MATCH_VQWBDOTAS_VV, MASK_VQWBDOTAS_VV, ...}
```

VS2 field 拆分



检查项	规则
vs2 group	只允许 v0/v8/v16/v24
ci	0,8,16,24,32,40,48,56
反汇编	打印 group base 与 ci*8

指令编码：使用 OP-VE major opcode 0x77

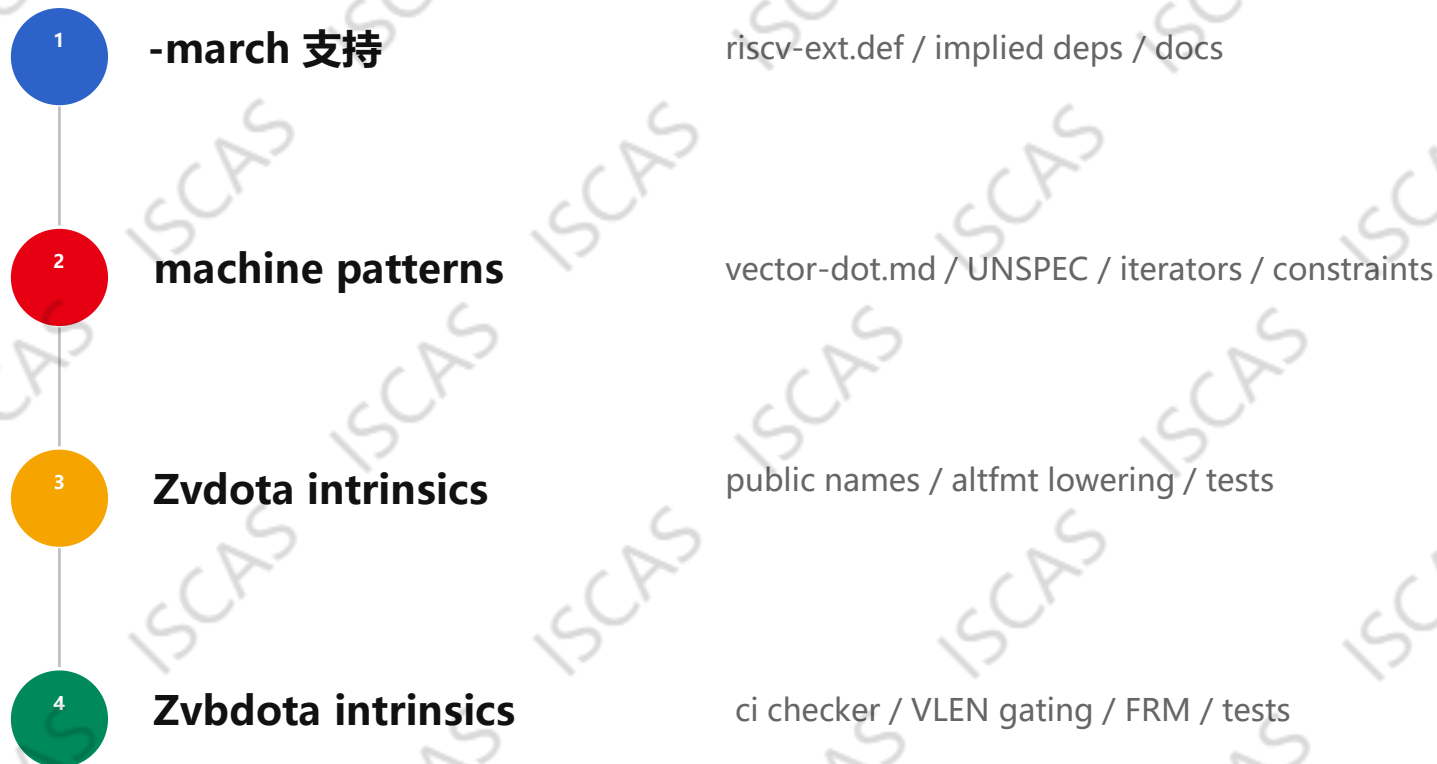
Zvdota/Zvbdota 使用新的 0x77 major opcode，并复用 ivv/fvv 风格 funct3/funct6 组合。

Family	Instruction	funct6	funct3	依赖
Zvdota	vqwdotau.vv	0x26	0	Zvqwdota8i/16i
Zvdota	vqwdotas.vv	0x27	0	Zvqwdota8i/16i
Zvdota	vfwdots.vv	0x24	1	Zvfwdots16bf
Zvdota	vfqwdota(.alt).vv	0x26/0x27	1	Zvfqwdota8f
Zvbdota	vqwbdotau/s.vv	0x2e/0x2f	0	Zvqwbdota8i/16i
Zvbdota	vfwbdots.vv	0x2c	1	Zvfwbdots16bf
Zvbdota	vfqwbdota(.alt).vv	0x2e/0x2f	1	Zvfqwbdota8f
Zvbdota	vfbdots.vv	0x2b	1	Zvfbdots32f

```
MATCH_VQWDOTAU_VV = 0x98000077
MATCH_VQWBDOTAU_VV = 0xb8000077
MASK = 0xfc00707f
```

工具链意义：不能按普通 RVV OP-V 0x57 处理；
assembler/disassembler 需要新增 match/mask。

GCC 实现结构：分层推进



原则：当前系列只提供显式 intrinsic 支持；自动向量化后续独立 RFC。

GCC machine pattern: destructive accumulator + altfmt

核心：vd 既是输入 accumulator，也是输出；altfmt 通过 pattern operand 显式传入。

```
(set (match_operand:<ACC> 0 "register_operand" "&vr")
  (if_then_else:<ACC>
    (unspec:<VM> [...] UNSPEC_VPREDICATE)
    (unspec:<ACC>
      [(match_operand:SRC 3 "register_operand" "vr")
       (match_operand:SRC 4 "register_operand" "vr")
       (match_operand:<ACC> 2 "register_operand" "0")
       (match_operand 9 "const_int_operand" "i")]
      ZVQWDOT)
    (match_dup 2)))
```

实现要点

- "&vr" + "0"：保守处理 overlap;
- altfmt 不再由 UNSPEC 推导，而是显式 operand，支持 integer/FP8 双维度映射
- BF16 pattern 固定 ALTFMT_ALT; vfbdoa 单独走 FRM
- 后续可迁移到 dependent constraints 提升寄存器分配灵活性

```
set_attr "altfmt"
  (symbol_ref "INTVAL (operands[9])")
```

RVV Intrinsic: public API 必须对齐 intrinsic doc

public name 反映类型组合; backend lowering 再选择真实 asm mnemonic.

Family	public naming scheme	示例
Zvdota	{OP}_vv_{VS2}_{VS1}_{VD}	__riscv_vqwdota_vv_u8mf4_i8mf4_i32m1
Zvbdota	{OP}_vv_{VS2_GROUP}_{VS1}_{VD}	__riscv_vqwbddota_vv_u8m8_i8m1_i32m8
vfbddota	vfbddota_vv_{VD}(_rm)	__riscv_vfbddota_vv_f32m8_rm

为什么不暴露 vqwdotau / vfqwdota_alt?

- intrinsic doc 使用统一 OP 名称, 由 VS2/VS1/VD type suffix 区分组合
- u/s、.alt 是 ISA mnemonic 的编码选择, 不是 public API base name
- GCC shape 负责把类型组合映射到 UNSPEC 与 altfmt

GCC 实现位置

- riscv-vector-builtins-functions.def: 注册 OP 与 type matrix
- riscv-vector-builtins-shapes.cc: 生成名称与参数检查
- riscv-vector-builtins-bases.cc: expand 到 machine pattern

Lowering 规则: mnemonic 只看 VS2, altfmt 只看 VS1

这个设计避免把 us/混合格式变体错误 lower 到 signed mnemonic.

整数 public type	asm mnemonic	altfmt
vs2 unsigned, vs1 unsigned	vqwdotau / vqwbdotau	0
vs2 unsigned, vs1 signed	vqwdotau / vqwbdotau	1
vs2 signed, vs1 unsigned	vqwdotas / vqwbdotas	0
vs2 signed, vs1 signed	vqwdotas / vqwbdotas	1

FP8 public type	asm mnemonic	altfmt
vs2 E4M3, vs1 E4M3	vfqwdota / vfqwbdota	0
vs2 E4M3, vs1 E5M2	vfqwdota / vfqwbdota	1
vs2 E5M2, vs1 E4M3	vfqwdota.alt / vfqwbdota.alt	0
vs2 E5M2, vs1 E5M2	vfqwdota.alt / vfqwbdota.alt	1

```
int unspec = vs2_unsigned_p ? UNSPEC_VQWDOTAU : UNSPEC_VQWDOTAS;  
unsigned altfmt = vs1_signed_p ? ALTFMT_ALT : ALTFMT_NONE;
```

测试检查: Zvdota vqwdotau/vqwdotas = 4/4; Zvbdota vqwbdotau/vqwbdotas = 4/4.

合法性检查：ci immediate 与 accumulator VLEN gating

ci immediate

- GCC built-in 层要求 compile-time constant
- 合法值：0, 8, 16, 24, 32, 40, 48, 56
- assembler 也会检查，但 GCC 侧可给出更早、更清晰的错误信息

```
if (ci < 0 || ci > 56 || ci % 8 != 0)
    error("constant multiple of 8 in range [0, 56]");
```

accumulator VLEN/LMUL gating

Accum EEW	C type LMUL	min VLEN
32-bit	m1	256
32-bit	m2	128
32-bit	m4	64
32-bit	m8	-
64-bit	m1	512
64-bit	m2	256
64-bit	m4	128
64-bit	m8	64

原因：vd EMUL = $\text{ceil}(8 \times \text{EEW} / \text{VLEN})$ 。GCC mode 必须覆盖实际 architectural register group。

测试验证：从汇编到 intrinsic 的闭环

Binutils

- gas: zvdota/zvbdota 主测试 + 子扩展单独测试
- objdump: mnemonic、mask、ci 反汇编一致
- readelf -A: Tag_RISCV_arch 中 extension 展开合理

GCC

- all-gcc 与 rvv/base/zvdota*.c、zvbdota*.c 回归通过
- scan-assembler 检查真实 mnemonic 与 e8alt/e16alt
- negative tests: no-rm、ci-invalid、VLEN/LMUL gating

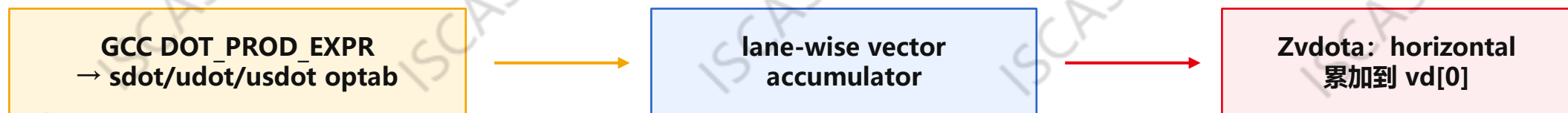
```
make check-gcc RUNTESTFLAGS="riscv.exp=zvdota*"
make check-gcc RUNTESTFLAGS="riscv.exp=zvbdota*"
make check-binutils
```

```
scan: vqwdotau.vv 4 / vqwdotas.vv 4
scan: vqwbdotau.vv 4 / vqwbdotas.vv 4
```

验证目标：public intrinsic 名称对齐 doc；backend 输出对齐 ISA mnemonic；assembler 接受并正确编码。

为什么当前不做自动向量化?

现有 GCC dot-product vectorizer 语义与 Zvdota/Zvbdota 不精确匹配。



当前结论

- 不把 Zvdota 硬接到 sdot_prod/udot_prod/usdot_prod, 避免 semantic mismatch
- 如果未来做 Zvdota autovec, 需要新增 horizontal dot reduction 的 IR/optab/IFN
- Zvbdota 更依赖 GEMM blocking、vs2_group、ci 和 VLEN gating, 适合后续 target-specific RFC

当前 patch: intrinsics-only

- Zvdota/Zvbdota扩展背景与设计
 - Horizontal dot-product 语义
 - batched dot-product
 - 数据格式、vtype.altfmt、ci immediate
- GNU工具链支持方案
 - Binutils 支持: -march、opcode、operand、tests
 - GCC 支持: machine pattern、RVV intrinsic、合法性检查
- Upstream进展与后续工作

Upstream 进展

目前已在RuyiSDK仓库中实现了Zvdota/Zvbdota的支持

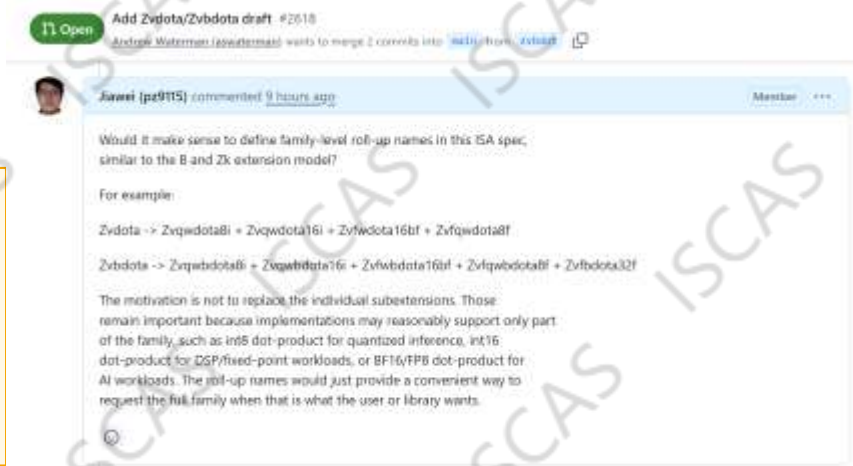
gcc: <https://github.com/ruyisdk/riscv-gcc/tree/zvdota>
binutils: <https://github.com/ruyisdk/riscv-binutils/tree/zvdota>

Reviewer / 主题	反馈	处理策略
Jan / binutils	-march 太长; 建议 roll-up	接受: zvdota/zvbdota roll-up + spec PR 询问
Jan / .insn	OP-VE ivv/fvv 会影响 .insn 设计	本系列只加 named opcode, .insn 单独讨论
Jeff / GCC pattern	需要更强 extension gate; spec frozen?	显式化条件; cover letter 说明状态
Robin / overlap	earlyclobber conservatively correct	当前保留; 后续 dependent constraints
Robin / iterators	尽量复用/按属性命名	重构 iterator 命名, 避免 instruction-specific

Roll-up 名称:

问题 —— 完整 family 的 -march 很长，测试和用户命令行可读性差

方案 —— zvdota / zvbdota 作为 fixed dependency expansion，子扩展仍可单独使用



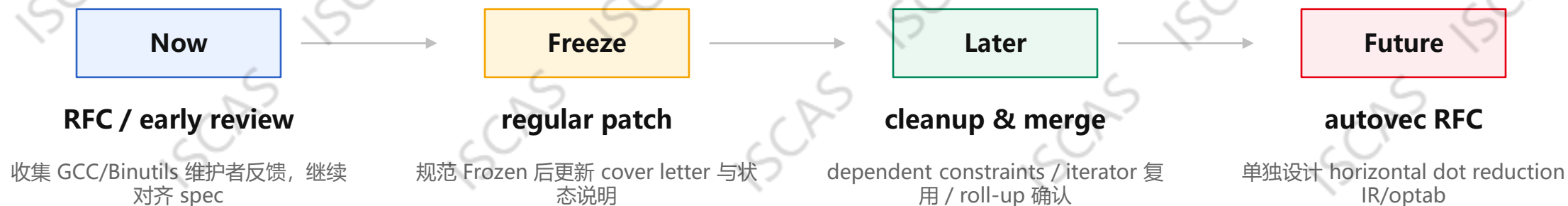
zvdota -> zvqwdota8i + zvqwdota16i + zvfwdota16bf + zvfqwdota8f

zvbdota -> zvqwbdots8i + zvqwbdots16i + zvfwbdots16bf + zvfqwbdots8f + zvfbdots32f

维度	individual subextension	roll-up
硬件裁剪	高：可只实现 int8 或 BF16	低：表示完整 family
用户命令行	长，容易漏写	短，可读性高
ELF/arch 属性	表达精确能力	表达 full-family 能力
未来扩展	可继续新增	应固定 v1.0 expansion

注意：不应把单个子扩展反向提升为 roll-up；避免 toolchain-only alias.

后续工作：从 RFC 到可合入支持



规格状态与发送策略

- ISA PR #2618 当前仍以 draft/RFC 推进；按 RVS track 等待 Freeze 确认
- GNU 上游通常要求 frozen/ratified；冻结前以 RFC/early review 发送更稳
- roll-up 若被 spec 接受，可避免 toolchain-only aliases

Takeaway

- Zvdota/Zvbdota 是面向 AI/DSP/GEMM 的重要 vector dot-product family
- 工具链实现难点集中在 altfmt、ci、destructive accumulator、intrinsic API 命名
- 当前最现实路径：intrinsics-first, autovec-later；spec/toolchain 并行迭代

Zvdota草案: <https://github.com/riscv/riscv-isa-manual/pull/2618>

Intrinsic文档: <https://github.com/riscv-non-isa/riscv-rvv-intrinsic-doc/blob/main/doc/rvv-intrinsic-spec.adoc#zvdota-dot-product-instructions>

草案进展追踪: <https://riscv.atlassian.net/browse/RVS-3925>

GCC支持: <https://github.com/ruyisdk/riscv-gcc/tree/zvdota>

Binutils支持: <https://github.com/ruyisdk/riscv-binutils/tree/zvdota>

工具链上游讨论情况:

<https://patchwork.sourware.org/project/gcc/list/?series=62097&state=%2A&archive=both>

<https://patchwork.sourware.org/project/binutils/list/?series=62029&state=%2A&archive=both>

交流合作: jiawei@iscas.ac.cn

感谢聆听!